

Project 1 – Search

Kylie Clark, Madison Ostinowsky

CMPSC 442

Q 1.1

See solution_q1.py.

Q1.2

1. Solution path and number of node expansions for DFS and BFS:

	Solution Path	Node Expansions
DFS	[3, 3, 0, 0, 'L'] -> [2, 2, 1, 1, 'R'] -> [3, 2, 0, 1, 'L'] -> [3, 0, 0, 3, 'R'] -> [3, 1, 0, 2, 'L'] -> [1, 1, 2, 2, 'R'] -> [2, 2, 1, 1, 'L'] -> [0, 2, 3, 1, 'R'] -> [0, 3, 3, 0, 'L'] -> [0, 1, 3, 2, 'R'] -> [0, 2, 3, 1, 'L'] -> [0, 0, 3, 3, 'R']	11
BFS	[3, 3, 0, 0, 'L'] -> [3, 1, 0, 2, 'R'] -> [3, 2, 0, 1, 'L'] -> [3, 0, 0, 3, 'R'] -> [3, 1, 0, 2, 'L'] -> [1, 1, 2, 2, 'R'] -> [2, 2, 1, 1, 'L'] -> [0, 2, 3, 1, 'R'] -> [0, 3, 3, 0, 'L'] -> [0, 1, 3, 2, 'R'] -> [1, 1, 2, 2, 'L'] -> [0, 0, 3, 3, 'R']	14

2. The main tradeoffs between DFS and BFS are the paths they choose and space complexity. BFS searches level by level and evaluates all paths on the level before going to the next. Because there is a cost of 1 for each action, BFS guarantees that the first path reaching the goal state has the minimal possible number of actions. Conversely, DFS chooses a path and takes it to the very end before backtracking to the next path. DFS does use less space than BFS because it only has to remember the current branch it is on. However, DFS does not guarantee an optimal path because it returns the first path it finds that hits the goal state, no matter the length.

The node expansion counts show that fewer expansions can make a search faster in a specific run, but does not guarantee completeness or optimality. In this simulation, DFS reached the goal state with fewer expansions (11 nodes), but this efficiency happened by chance rather than a reliable plan. BFS had to expand 14 nodes because it explored level by level before going deeper, verifying that no shorter path existed. This approach ensures BFS is complete and optimal as long as a solution exists. It guarantees the shortest path, while DFS does not. Therefore, while DFS happened to be more efficient in this run, BFS will always provide the shortest solution.

Q 2.1

See `solution_q2.py`.

Q 2.2

1. Solution path, cost, and number of node extensions for UCS Cost Model A and UCS Cost Model B:

	Solution Path	Cost	Node Expansions
Cost Model A	[3, 3, 0, 0, 'L'] -> [3, 1, 0, 2, 'R'] -> [3, 2, 0, 1, 'L'] -> [3, 0, 0, 3, 'R'] -> [3, 1, 0, 2, 'L'] -> [1, 1, 2, 2, 'R'] -> [2, 2, 1, 1, 'L'] -> [0, 2, 3, 1, 'R'] -> [0, 3, 3, 0, 'L'] -> [0, 1, 3, 2, 'R'] -> [0, 2, 3, 1, 'L'] -> [0, 0, 3, 3, 'R']	23	15
Cost Model B	[3, 3, 0, 0, 'L'] -> [2, 2, 1, 1, 'R'] -> [3, 2, 0, 1, 'L'] -> [3, 0, 0, 3, 'R'] -> [3, 1, 0, 2, 'L'] -> [1, 1, 2, 2, 'R'] -> [2, 2, 1, 1, 'L'] -> [0, 2, 3, 1, 'R'] -> [0, 3, 3, 0, 'L'] -> [0, 1, 3, 2, 'R'] -> [0, 2, 3, 1, 'L'] -> [0, 0, 3, 3, 'R']	17	15

2. For the most part, both UCS (for both cost models) and BFS follow a similar sequence of actions, with small differences at the start and the end of their paths. The path length of BFS is 12 states, and the costs of the two UCS models are 23 and 17, so the path length of BFS is shorter than the cost of UCS is large. Both UCS cost models have node expansions of 15 while BFS has a node expansion of 14, meaning BFS leads to fewer node expansions, but not by much.
3. With varying costs, it is possible that the shortest path has the highest cost, and that the longest path has the lowest cost. With equal costs for all actions, UCS

essentially is just BFS, giving the same result because, in this case, the shortest path will have the lowest cost. When costs are not all equal, UCS is needed to determine the path with the lowest cost. Choosing between lowest cost and shortest path is entirely up to the situation and the user. However, because UCS with unequal costs is concerned with cost and not path length, it will likely explore more nodes to determine the true, lowest cost path.

Q 3.1

See solution_q3.py.

Q 3.2

- A. Heuristic 1 is admissible because it is only concerned with the cost of moving the people on the left to the right, not with the constraints that there are missionaries and cannibals and not considering that the boat must also move 1 person from right to left to get the boat back to collect more people. The largest Heuristic 1 will ever be in this situation is 9, which does not consider that the boat will always need to travel with at least one person from right to left, so the cost will always be higher than this heuristic.

Heuristic 2 is admissible because it is a scaled down version of Heuristic 1, meaning it will always be smaller than Heuristic 1, and if Heuristic 1 will always be smaller than the actual cost, then it is clear that Heuristic 2 will also always be smaller than the actual cost. The largest Heuristic 2 will ever be in this situation is 3, which is lower than the cost to even send all the missionaries across, let alone all the people, so the cost will always be higher than this heuristic.

B. Solution path, cost, and number of node extensions for A* with Heuristic 1, Heuristic 2, and proposed Heuristic 3:

Heuristic	Solution Path	Total Cost	Node Expansions	Same Cost as UCS Cost Model A?
h_1	[3, 3, 0, 0, 'L'] -> [3, 1, 0, 2, 'R'] -> [3, 2, 0, 1, 'L'] -> [3, 0, 0, 3, 'R'] -> [3, 1, 0, 2, 'L'] -> [1, 1, 2, 2, 'R'] -> [2, 2, 1, 1, 'L'] -> [0, 2, 3, 1, 'R'] -> [0, 3, 3, 0, 'L'] -> [0, 1, 3, 2, 'R'] -> [0, 2, 3, 1, 'L'] -> [0, 0, 3, 3, 'R']	23	13	Yes
h_2	[3, 3, 0, 0, 'L'] -> [3, 1, 0, 2, 'R'] -> [3, 2, 0, 1, 'L'] -> [3, 0, 0, 3, 'R'] -> [3, 1, 0, 2, 'L'] -> [1, 1, 2, 2, 'R'] -> [2, 2, 1, 1, 'L'] -> [0, 2, 3, 1, 'R'] -> [0, 3, 3, 0, 'L'] -> [0, 1, 3, 2, 'R'] -> [0, 2, 3, 1, 'L'] -> [0, 0, 3, 3, 'R']	23	14	Yes
h_3	[3, 3, 0, 0, 'L'] -> [3, 1, 0, 2, 'R'] -> [3, 2, 0, 1, 'L'] -> [3, 0, 0, 3, 'R'] -> [3, 1, 0, 2, 'L'] -> [1, 1, 2, 2, 'R'] -> [2, 2, 1, 1, 'L'] -> [0, 2, 3, 1, 'R'] -> [0, 3, 3, 0, 'L'] -> [0, 1, 3, 2, 'R'] -> [0, 2, 3, 1, 'L'] -> [0, 0, 3, 3, 'R']	23	13	Yes

Heuristic 1 is more efficient, with 13 expansions, because heuristic 2 is essentially the weaker version of heuristic 1. Heuristic 1 adds a strong repulsion (3 times what is added with heuristic 2) to states that are farther from the goal, causing the algorithm to be greedier and more likely to select a node with a state closer to the goal.

C. Proposed heuristic: Estimate the cost for everyone who still needs to cross, and if the boat is on the wrong side, add the minimum cost required to bring it back.

This heuristic considers which side the boat is on. With h_1 , two states with the same number of missionaries and cannibals on each side would receive the same estimate, regardless of which side the boat is on. With h_3 , these states are treated differently. If the boat is on the right while people remain on the left, someone must return the boat before those people can continue toward the goal state. Since the cheapest possible return trip has a cost of 1, h_3 adds 1 in this situation.

$$h_3(s) = \begin{cases} 2M_{\text{left}} + 1C_{\text{left}} + 1 & \text{if boat is on right and people remain on left} \\ 2M_{\text{left}} + 1C_{\text{left}} & \text{otherwise} \end{cases}$$

This dominates h_1 because it uses the same estimate as h_1 , but also accounts for the location of the boat. Therefore, $h_3(s) \geq h_1(s)$ for every state, and $h_3(s) > h_1(s)$ whenever the boat is on the right and people remain on the left.

This is admissible because it never overestimates the remaining cost. The original h_1 provides a lower bound on the cost of moving the remaining people to the right bank. An additional 1 is added when it must go back to the left bank before the right bank because the goal state is always on the right bank. This is the lowest cost for that trip that was not being taken account for in h_1 . Therefore, h_3 remains a lower bound on the actual cost required to reach the goal.

*Resulting table in part B

Q3.3 Extra Credit

To be consistent, the estimate should not decrease by more than the actual cost of moving from one state to the other.

Proofs:

Heuristic h_1 – Passenger Weight Remaining

$$h_1(s) = 2M_{\text{left}} + C_{\text{left}}$$

For any move from state s to state t , to be consistent, it must be true that:

$$h_1(s) - h_1(t) \leq \text{cost}(s \rightarrow t)$$

Completing a move from left to right, there are m missionaries and c cannibals that cross from left to right. So, the heuristic will decrease by: $h_1(s) - h_1(t) = 2m + c$

The actual cost of this move under Cost Model A is: $\text{cost}(s \rightarrow t) = 2m + c$

$$\text{So, } 2m + c \leq 2m + c$$

The condition for consistency is satisfied for left to right.

Now, from right to left, M_{left} and/or C_{left} increase, so $h_1(s) - h_1(t) < 0$

Since every move has a positive cost: $h_1(s) - h_1(t) \leq \text{cost}(s \rightarrow t)$

Therefore, h_1 is consistent.

Heuristic h_2 – Trip-Packing Lower Bound

$$h_2 = \left\lceil \frac{2M_{\text{left}} + C_{\text{left}}}{3} \right\rceil$$

Completing a move from left to right, $2M_{\text{left}} + C_{\text{left}}$ decreases by the cost of the passengers moved, so: $2m + c$

Because it is divided by 3 and then rounded up, the decrease in h_2 should not be more than the cost it actually was to move: $h_2(s) - h_2(t) \leq 2m + c$

$$\text{and } 2m + c = \text{cost}(s \rightarrow t)$$

$$\text{So, } h_2(s) - h_2(t) \leq \text{cost}(s \rightarrow t)$$

For the right to left move, the left side amount is what increases, so $h_2(t) \geq h_2(s)$

$$\text{So, } h_2(s) - h_2(t) \leq 0$$

Therefore, h_2 is consistent.

Heuristic h_3 – Boat Location

$$h_3(s) = \begin{cases} 2M_{\text{left}} + 1C_{\text{left}} + 1 & \text{if boat is on right and people remain on left} \\ 2M_{\text{left}} + 1C_{\text{left}} & \text{otherwise} \end{cases}$$

Going from left to right it decreases by the cost $2m + c$. If people remain on the left, then the boat ends on the right. So, h_3 will gain 1. The decrease in the heuristic cannot be greater than the move cost so, $h_3(s) - h_3(t) \leq 2m + c$,

Therefore: $h_3(s) - h_3(t) \leq \text{cost}(s \rightarrow t)$

Going from right to left, the boat is not penalized the 1, but the passenger moved back increases the $2M_{\text{left}} + C_{\text{left}}$. The smallest return would have to be one cannibal with a cost of 1. So, this offsets that added 1 penalty in the left to right. Therefore, h_3 cannot decrease by more than the actual cost to move.

So, $h_3(s) - h_3(t) \leq \text{cost}(s \rightarrow t)$

So h_3 is consistent